SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	**5**
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	**5**
3	Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	**5**
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	**5**
5	Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	**5**
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	**5**
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	**5**

8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	**5**
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	**5**
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	**5**

Code of Conduct:

I CH. Durga Prasad (192425305) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Ch. Durga Prasad

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1.For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

For the relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

we identify the functional dependencies based on the meaning of the attributes.

1. Functional Dependencies

Assume:

- SID uniquely identifies a student.
- CID uniquely identifies a course.
- A student's grade is determined by the student and the course.

Therefore:

1. SID → SName

   A Student ID determines the Student Name.
2. CID → CName

   A Course ID determines the Course Name.
3. CID → Instructor

   A Course ID determines the Instructor.
4. (SID, CID) → Grade

   A student's grade is determined by the combination of Student ID and Course ID.

So the main functional dependencies are:

SID → SName

CID → CName, Instructor

SID, CID → Grade

2. Candidate Key

Consider the combination (SID, CID):

- SID identifies the student.
- CID identifies the course.
- Together, (SID, CID) determines:
  - SName through SID
  - CName and Instructor through CID

- Grade through (SID, CID)

Thus:

(SID, CID) → SName, CName, Instructor, Grade

Neither SID nor CID alone can determine all attributes.

Therefore, the candidate key is:

Candidate Key = {SID, CID}

Final Answer

Functional Dependency	Explanation
SID → SName	Student ID determines student name
CID → CName	Course ID determines course name
CID → Instructor	Course ID determines instructor
SID, CID → Grade	Student-course combination determines grade

Candidate Key: {SID, CID}

Primary Key: (SID, CID) can be selected as the composite primary key.

2. Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Converting an Unnormalized Relation into 1NF

Introduction

First Normal Form (1NF) is the first step in database normalization. A relation is in **1NF** when:

- Each attribute contains **atomic (single) values**.
- There are **no repeating groups or multi-valued attributes**.
- Each row represents a unique record.

Example: Unnormalized Relation

Consider the following **STUDENT_COURSE** relation:

SID	SName	Courses	Instructor
S101	Durga	DBMS, Python	Dr. Kumar, Dr. Ravi
S102	Dharanish	Java, DBMS	Dr. Arun, Dr. Kumar
S103	Sabesh	Python	Dr. Ravi

This relation is **not in 1NF** because the Courses and Instructor attributes contain multiple values in a single cell.

For example:

S101 → Courses = DBMS, Python

This violates the atomicity requirement of 1NF.

Step 1: Identify the Repeating Groups

The repeating attributes are:

- Courses
- Instructor

Each student can have multiple courses and instructors.

Therefore, these multi-valued attributes need to be separated into individual rows.

Step 2: Remove Repeating Groups

Instead of storing multiple courses in one row, create **one row for each student-course combination**.

Converted Relation

SID	SName	Course	Instructor
S101	Durga	DBMS	Dr. Kumar
S101	Durga	Python	Dr. Ravi
S102	Dharanish	Java	Dr. Arun
S102	Dharanish	DBMS	Dr. Kumar
S103	Sabesh	Python	Dr. Ravi

Now every cell contains only **one value**.

Step 3: Identify the Key

A student can enroll in multiple courses, so SID alone cannot uniquely identify a row.

For example:

SID	Course
S101	DBMS
S101	Python

Therefore, we use the combination:

(SID, Course)

as the **candidate key**.

Step 4: Verify 1NF

The resulting relation is:

STUDENT_COURSE(SID, SName, Course, Instructor)

with candidate key:

(SID, Course)

It satisfies 1NF because:

1. Every attribute contains a **single atomic value**.
2. There are **no repeating groups**.
3. Each row represents a **single student-course enrollment**.
4. (SID, Course) uniquely identifies each record.

Final 1NF Relation

STUDENT_COURSE

SID	SName	Course	Instructor
S101	Durga	DBMS	Dr. Kumar
S101	Durga	Python	Dr. Ravi
S102	Dharanish	Java	Dr. Arun
S102	Dharanish	DBMS	Dr. Kumar
S103	Sabesh	Python	Dr. Ravi

Candidate Key = (SID, Course)

Conclusion

The conversion from an unnormalized relation to **1NF** is achieved by **removing repeating and multi-valued attributes and representing each value in a separate row**. The resulting table contains only atomic values and can then be further normalized into **2NF and 3NF**.

3. Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies

Given Relation

R(A, B, C, D, E)

Functional Dependencies:

1. A → BC

2. BC → D
3. D → E

We need to analyze the relation and determine whether it violates 2NF.

Step 1: Find the Candidate Key

Start with A^+:

- A → B, C
- BC → D
- D → E

Therefore:

$A^+ = \{A, B, C, D, E\}$

Since A determines all attributes:

Candidate Key = A

So the candidate key contains only one attribute.

Step 2: Check for Partial Dependencies

A relation violates 2NF when a non-prime attribute depends on only a proper subset of a composite candidate key.

Here:

Candidate Key = {A}

Since the key contains only one attribute, it has no proper subset.

Therefore, partial dependency is impossible.

The dependencies are:

FD	Type
A → B, C	Full dependency
BC → D	Not a partial dependency
D → E	Not a partial dependency

Hence, R is already in 2NF.

Step 3: 2NF Decomposition

Since there are no partial dependencies, no decomposition is required to achieve 2NF.

Therefore:

R(A, B, C, D, E)

is already in 2NF.

Why?

The only candidate key is A, and because it is a single-attribute key, there cannot be any partial dependency.

Important Observation: Transitive Dependencies

Although the relation is in 2NF, it is not in 3NF because there are transitive dependencies:

A → BC → D → E

For example:

- A → B, C
- BC → D
- D → E

Thus, A indirectly determines E through D.

If the question were asking for 3NF decomposition, we could decompose it into:

1. R1(A, B, C)
2. R2(B, C, D)
3. R3(D, E)

But for 2NF, this decomposition is not necessary.

Final Answer

Candidate Key = A

Partial Dependencies = None

2NF Decomposition = Not required

R(A,B,C,D,E) is already in 2NF.

Note: The relation has transitive dependencies, so further decomposition would be needed for 3NF, not for 2NF.

4.Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

3NF Decomposition of EMPLOYEE Relation

Given relation:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Assume the following functional dependencies:

1. EmpID → EmpName, DeptID
2. DeptID → DeptName, ManagerID

Here, DeptID → DeptName, ManagerID creates a transitive dependency:

EmpID → DeptID → DeptName, ManagerID

Therefore, the relation is not in 3NF.

Step 1: Find the Candidate Key

Since:

EmpID → EmpName, DeptID

and

DeptID → DeptName, ManagerID

we get:

$EmpID^{+}$ = {EmpID, EmpName, DeptID, DeptName, ManagerID}

Therefore:

Candidate Key = EmpID

Step 2: Identify the Transitive Dependency

We have:

EmpID → DeptID

and

DeptID → DeptName, ManagerID

Therefore:

EmpID → DeptName, ManagerID

through DeptID.

This is a transitive dependency, which violates 3NF.

Step 3: Decompose the Relation

Separate the department-related attributes into a new relation.

Relation 1: EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID)

EmpID	EmpName	DeptID
E101	Durga	D01
E102	Dharanish	D02
E103	Sabesh	D01

Primary Key: EmpID

Foreign Key: DeptID

Relation 2: DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID)

DeptID	DeptName	ManagerID
D01	Computer Science	M101
D02	Artificial Intelligence	M102

Primary Key: DeptID

Step 4: Verify 3NF

EMPLOYEE

EmpID → EmpName, DeptID

- EmpID is the candidate key.
- All non-key attributes depend directly on the key.
- No transitive dependency.

Therefore, EMPLOYEE is in 3NF.

DEPARTMENT

DeptID → DeptName, ManagerID

- DeptID is the candidate key.
- All non-key attributes depend directly on DeptID.
- No transitive dependency.

Therefore, DEPARTMENT is in 3NF.

Final Decomposition

EMPLOYEE(EmpID, EmpName, DeptID)

|

| DeptID

↓

DEPARTMENT(DeptID, DeptName, ManagerID)

Relation	Primary Key	Foreign Key
EMPLOYEE	EmpID	DeptID
DEPARTMENT	DeptID	—

Conclusion

The original relation has the transitive dependency:

EmpID → DeptID → DeptName, ManagerID

To achieve 3NF, we decompose it into:

EMPLOYEE(EmpID, EmpName, DeptID)

DEPARTMENT(DeptID, DeptName, ManagerID)

This removes the transitive dependency, reduces data redundancy, and prevents update, insertion, and deletion anomalies.

5.Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.

BCNF Verification and Decomposition

Given relation:

R(A, B, C, D)

Functional Dependencies:

1. AB → C
2. C → D
3. D → A

We need to determine whether the relation is in BCNF and, if not, decompose it.

Step 1: Find the Candidate Keys

We calculate the closures.

Closure of AB

From AB → C:

$AB^+ = \{A, B, C\}$

From C → D:

$AB^+ = \{A, B, C, D\}$

Therefore:

AB is a candidate key.

Closure of BC

From C → D:

B, C → D

From D → A:

B, C, D → A

Therefore:

$BC^+ = \{A, B, C, D\}$

So:

BC is a candidate key.

Closure of BD

From D → A:

BD → A

Now we have A and B, so:

AB → C

Therefore:

$BD^+ = \{A, B, C, D\}$

So:

BD is a candidate key.

Thus, the candidate keys are:

AB, BC, BD

Step 2: Check BCNF

A relation is in BCNF if, for every non-trivial functional dependency:

X → Y, X must be a superkey.

Check each FD:

FD	Is determinant a superkey?	BCNF?
AB → C	Yes, AB is a candidate key	✓
C → D	No	✗
D → A	No	✗

Therefore:

R is NOT in BCNF.

The violating dependencies are:

C → D and D → A

Step 3: Decompose Using C → D

Choose the violating FD:

C → D

Decompose R into:

R1(C, D)

FD:

C → D

Here, C is the candidate key, so R1 is in BCNF.

R2(A, B, C)

The remaining relation is:

R2(A, B, C)

The FD AB → C holds, and AB is a candidate key of R2.

Therefore, R2 is also in BCNF.

Step 4: Final BCNF Relations

The decomposition is:

R1(C, D)

R2(A, B, C)

R1(C, D)

C	D
C1	D1
C2	D2

Candidate Key: C

FD: C → D

Therefore, R1 is in BCNF.

R2(A, B, C)

A	B	C
A1	B1	C1
A2	B2	C2

Candidate Key: AB

FD: AB → C

Therefore, R2 is in BCNF.

Final Answer

Original Relation:

R(A, B, C, D)

FDs:

AB → C

C → D

D → A

Candidate Keys:

AB, BC, BD

BCNF Violations:

C → D

D → A

BCNF Decomposition:

R1(C, D)

R2(A, B, C)

Conclusion

R(A,B,C,D) is not in BCNF because C → D and D → A have determinants that are not superkeys.

After decomposition:

R1(C,D) and R2(A,B,C)

both satisfy the BCNF condition.

6.Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

Aim:

To experimentally verify whether a decomposition of a relation is lossless-join using the tabular (Chase) algorithm.

Example

Consider the relation:

R(A, B, C)

with the functional dependency:

A → B

Decompose R into:

- $R_1(A, B)$
- $R_2(A, C)$

We need to verify whether this decomposition is lossless.

Step 1: Create the Chase Table

Create one row for each decomposed relation and one column for every attribute of R.

Initially:

Row	A	B	C
$R_1(A,B)$	a	b	c_1
$R_2(A,C)$	a	b_2	c

Here:

- a, b, c are distinguished symbols.
- b_2, c_1 are different symbols.

For attributes that belong to a decomposition relation, we use the same distinguished symbol as the original attribute.

Step 2: Apply the Functional Dependency

The given FD is:

$A \rightarrow B$

Both rows have the same value for A:

Row	A	B	C
R_1	a	b	c_1
R_2	a	b_2	c

Since:

A is the same in both rows

the FD $A \rightarrow B$ requires the B values to become equal.

Therefore, replace b_2 with b.

Updated Table

Row	A	B	C
R_1	a	b	c_1
R_2	a	b	c

Step 3: Check the Lossless-Join Condition

For a decomposition to be lossless, after applying all functional dependencies, at least one row must contain only the distinguished symbols:

a, b, c

The second row is:

A	B	C
a	b	c

Therefore, a complete distinguished row has been obtained.

Hence:

The decomposition is LOSSLESS.

Why Is It Lossless?

The common attribute between the two decomposed relations is:

$R_1 \cap R_2 = \{A\}$

And we have:

$A \rightarrow B$

Therefore:

$A \rightarrow AB$

which means the common attribute A functionally determines all attributes of R_1.

This guarantees that the decomposition:

$R(A,B,C) \rightarrow R_1(A,B), R_2(A,C)$

is lossless.

Chase Algorithm – General Procedure

The Chase algorithm can be performed using these steps:

1. Create a table with one row for each decomposed relation.
2. Create columns for all attributes of the original relation.
3. Assign the same distinguished symbols to attributes belonging to each decomposed relation.
4. Apply the given functional dependencies repeatedly.
5. Whenever two rows have the same values for the determinant of an FD, make their dependent attributes equal.

6. Continue until no further changes are possible.
7. If any row becomes completely distinguished, the decomposition is lossless.
8. Otherwise, the decomposition is lossy.

Final Result

Original Relation:

R(A, B, C)

FD:

$A \rightarrow B$

Decomposition:

R_1(A, B)

R_2(A, C)

Chase Result:

	A	B	C
R_1	a	b	c_1
R_2	a	b	c

Complete distinguished row exists:

a b c

Therefore:

✓ Decomposition is LOSSLESS

Conclusion

The tabular (Chase) algorithm experimentally verifies that the decomposition R(A,B,C) → R_1(A,B), R_2(A,C) is lossless, because application of the functional dependency $A \rightarrow B$ produces a row containing all distinguished symbols.

7. Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

4NF Decomposition of TEACHER

Given relation:

TEACHER(TID, Subject, Hobby)

Assume that a teacher can teach **multiple independent subjects** and can have **multiple independent hobbies**.

For example:

TID	Subject	Hobby
T01	DBMS	Reading
T01	DBMS	Music
T01	Python	Reading
T01	Python	Music

Here, the subjects and hobbies are **independent multi-valued attributes**.

Step 1: Identify the Multi-Valued Dependencies

The multi-valued dependencies are:

TID →→ Subject

and

TID →→ Hobby

This means:

- A teacher can have multiple subjects independently of their hobbies.
- A teacher can have multiple hobbies independently of their subjects.

For example:

T01 →→ Subject

means that the subjects taught by T01 are independent of the hobbies of T01.

Similarly:

T01 →→ Hobby

means that the hobbies of T01 are independent of the subjects taught by T01.

Step 2: Check for 4NF Violation

A relation is in **Fourth Normal Form (4NF)** if, for every non-trivial MVD:

X →→ Y, X must be a superkey.

In our relation:

TID →→ Subject

But TID alone is **not a superkey** of the original relation because it does not uniquely determine one Subject-Hobby combination.

Similarly:

TID →→ Hobby

violates the 4NF condition.

Therefore:

TEACHER(TID, Subject, Hobby) is NOT in 4NF.

Step 3: Decompose the Relation

For the MVD:

TID →→ Subject

decompose the relation into:

Relation 1

TEACHER_SUBJECT(TID, Subject)

TID	Subject
T01	DBMS
T01	Python

Relation 2

TEACHER_HOBBY(TID, Hobby)

TID	Hobby
T01	Reading
T01	Music

Step 4: Verify 4NF

TEACHER_SUBJECT

TEACHER_SUBJECT(TID, Subject)

Here:

TID →→ Subject

TID determines the complete set of subjects for a teacher.

TID is a key of this relation, so the 4NF condition is satisfied.

TEACHER_HOBBY

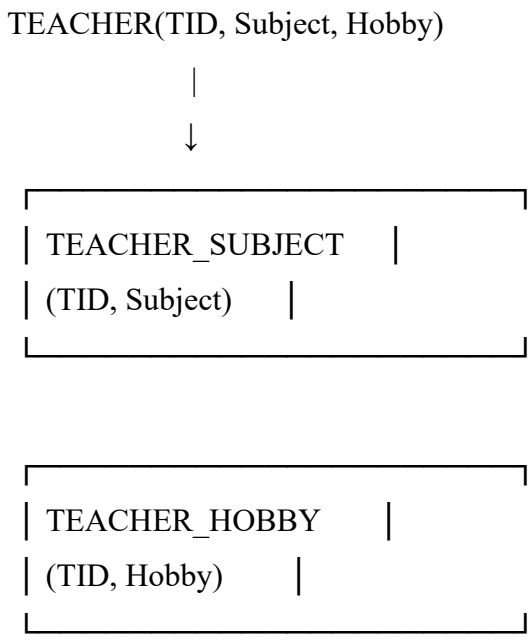
TEACHER_HOBBY(TID, Hobby)

Here:

TID →→ Hobby

TID is a key of this relation, so it also satisfies 4NF.

Final Decomposition



Final Answer

Multi-valued dependencies:

TID →→ Subject

TID →→ Hobby

4NF decomposition:

TEACHER_SUBJECT(TID, Subject)

TEACHER_HOBBY(TID, Hobby)

Conclusion

The original relation violates **4NF** because Subject and Hobby are independent multi-valued attributes of a teacher. Decomposing it into separate **TEACHER_SUBJECT** and **TEACHER_HOBBY** relations eliminates the multi-valued dependency and removes unnecessary redundancy.

8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

5NF (Project-Join Normal Form) Decomposition

Aim

To demonstrate how **join dependencies** can cause redundancy and decompose a relation into **5NF (Project-Join Normal Form)**.

Example Relation

Consider the relation:

SUPPLY(Supplier, Part, Project)

It records which supplier supplies which part to which project.

Suppose the following business rule exists:

A supplier can supply a part, a supplier can work on a project, and a part can be used in a project independently.

Original Relation

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

Here, the three attributes are independent.

Step 1: Identify the Join Dependency

The relation has a **join dependency**:

(Supplier, Part), (Supplier, Project), (Part, Project)

It can be written as:

(SP, SJ, PJ)

This means the original SUPPLY relation can be reconstructed by joining three smaller relations.

Step 2: Identify the Redundancy

Suppose:

Supplier-Part

Supplier	Part
S1	P1
S1	P2

Supplier-Project

Supplier	Project
S1	J1
S1	J2

Part-Project

Part	Project
P1	J1
P1	J2
P2	J1
P2	J2

The original relation contains all combinations:

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

If we store these combinations directly, information is repeated.

For example, the fact that **S1 supplies P1** is repeated for every project involving P1.

Similarly, the fact that **S1 works on J1** is repeated for every part supplied to J1.

This creates **redundancy**.

Step 3: Decompose the Relation

Decompose SUPPLY(Supplier, Part, Project) into three relations:

1. SUPPLIER_PART

SUPPLIER_PART(Supplier, Part)

Supplier	Part
S1	P1
S1	P2

2. SUPPLIER_PROJECT

SUPPLIER_PROJECT(Supplier, Project)

Supplier	Project
S1	J1
S1	J2

3. PART_PROJECT

PART_PROJECT(Part, Project)

Part	Project
P1	J1
P1	J2
P2	J1
P2	J2

Step 4: Reconstruct the Original Relation

The original relation can be obtained by joining:

SUPPLIER_PART ⋈ SUPPLIER_PROJECT ⋈ PART_PROJECT

The result is:

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

Thus, the decomposition preserves the required information.

Step 5: Verify 5NF

A relation is in **5NF** if every non-trivial **join dependency** is implied by the candidate keys of the relation.

The original relation has a non-trivial join dependency:

(SP, SJ, PJ)

Therefore, it can be decomposed into:

SUPPLIER_PART(Supplier, Part)

SUPPLIER_PROJECT(Supplier, Project)

PART_PROJECT(Part, Project)

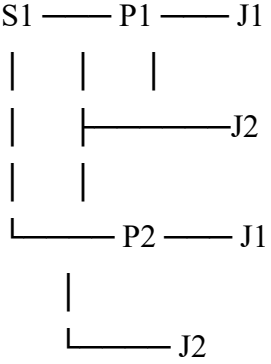
After decomposition, no further non-trivial join dependency exists that causes redundancy.

Hence, the decomposed relations are in **5NF**.

Before and After

Before 5NF

SUPPLY(Supplier, Part, Project)



The combinations cause repeated information.

After 5NF

SUPPLIER_PART

(Supplier, Part)

SUPPLIER_PROJECT

(Supplier, Project)

PART_PROJECT

(Part, Project)

Each independent relationship is stored only once.

Final Answer

Original Relation:

SUPPLY(Supplier, Part, Project)

Join Dependency:

(Supplier, Part), (Supplier, Project), (Part, Project)

5NF Decomposition:

SUPPLIER_PART(Supplier, Part)

SUPPLIER_PROJECT(Supplier, Project)

PART_PROJECT(Part, Project)

Conclusion

Join dependencies can cause redundancy when independent relationships are stored together in a single multi-attribute relation. Decomposing the relation into smaller relations based on its join dependency eliminates unnecessary repetition and produces a **5NF (Project-Join Normal Form)** design.

9.Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Normalizing a Poorly Designed College Database to 3NF

1. Poorly Designed Relation

Consider the following unnormalized/poorly designed relation:

COLLEGE(StudID, StudName, DeptID, DeptName, CourseID, CourseName, Instructor, Grade)

Example data:

StudID	StudName	DeptID	DeptName	CourseID	CourseName	Instructor	Grade
S101	Durga	D01	CSE	C01	DBMS	Kumar	A
S101	Durga	D01	CSE	C02	Python	Ravi	B+
S102	Dharanish	D01	CSE	C01	DBMS	Kumar	A+
S103	Sabesh	D02	AIML	C03	Machine Learning	Arun	A

Assume these functional dependencies:

1. StudID → StudName, DeptID
2. DeptID → DeptName
3. CourseID → CourseName, Instructor
4. (StudID, CourseID) → Grade

2. Identify the Candidate Key

A student can take multiple courses, and a course can have multiple students.

Therefore:

(StudID, CourseID) uniquely identifies a record.

So:

Candidate Key = (StudID, CourseID)

3. Identify the Anomalies

A. Insertion Anomaly

Suppose a new department is created:

D03 – Cyber Security

but no student has enrolled yet.

In the original table, we cannot easily insert the department information because StudID and course information are also required.

Similarly, if a new course C04 – Cloud Computing is introduced but no student has registered for it, the course information cannot be stored independently.

Problem: New department/course information depends on student enrollment.

B. Update Anomaly

Suppose the department name changes:

CSE → Computer Science and Engineering

The name appears in multiple rows:

StudID	DeptID	DeptName
S101	D01	CSE
S102	D01	CSE

Every occurrence must be updated.

If only one row is updated, the database contains inconsistent information.

Problem: The same information is stored repeatedly.

C. Deletion Anomaly

Suppose S103 is the only student enrolled in C03.

If S103's record is deleted, we also lose information about:

- Course C03

- Machine Learning
- Instructor Arun

Similarly, if the last student in a department is deleted, department information may also disappear.

Problem: Deleting enrollment information unintentionally deletes department/course information.

4. Convert to 1NF

The relation already contains atomic values, so it satisfies 1NF.

COLLEGE(
  StudID,
  StudName,
  DeptID,
  DeptName,
  CourseID,
  CourseName,
  Instructor,
  Grade
)

Primary/Candidate Key:

(StudID, CourseID)

5. Convert to 2NF

2NF requires that there are no partial dependencies on part of a composite key.

Our key is:

(StudID, CourseID)

But:

StudID → StudName, DeptID

StudName and DeptID depend only on StudID, not on the entire composite key.

Also:

CourseID → CourseName, Instructor

These attributes depend only on CourseID.

Therefore, partial dependencies exist.

Decompose into:

STUDENT

STUDENT(StudID, StudName, DeptID)

StudID	StudName	DeptID
S101	Durga	D01
S102	Dharanish	D01
S103	Sabesh	D02

Primary Key: StudID

COURSE

COURSE(CourseID, CourseName, Instructor)

CourseID	CourseName	Instructor
C01	DBMS	Kumar
C02	Python	Ravi
C03	Machine Learning	Arun

Primary Key: CourseID

ENROLLMENT

ENROLLMENT(StudID, CourseID, Grade)

StudID	CourseID	Grade
S101	C01	A
S101	C02	B+
S102	C01	A+
S103	C03	A

Primary Key: (StudID, CourseID)

6. Convert to 3NF

Now check for transitive dependencies.

In STUDENT:

StudID → DeptID

and:

DeptID → DeptName

Therefore:

StudID → DeptID → DeptName

This is a transitive dependency.

So we separate department information.

DEPARTMENT

DEPARTMENT(DeptID, DeptName)

DeptID	DeptName
D01	CSE
D02	AIML

Primary Key: DeptID

Final 3NF Schema

1. STUDENT

STUDENT(StudID, StudName, DeptID)

- Primary Key: StudID
- Foreign Key: DeptID

2. DEPARTMENT

DEPARTMENT(DeptID, DeptName)

- Primary Key: DeptID

3. COURSE

COURSE(CourseID, CourseName, Instructor)

- Primary Key: CourseID

4. ENROLLMENT

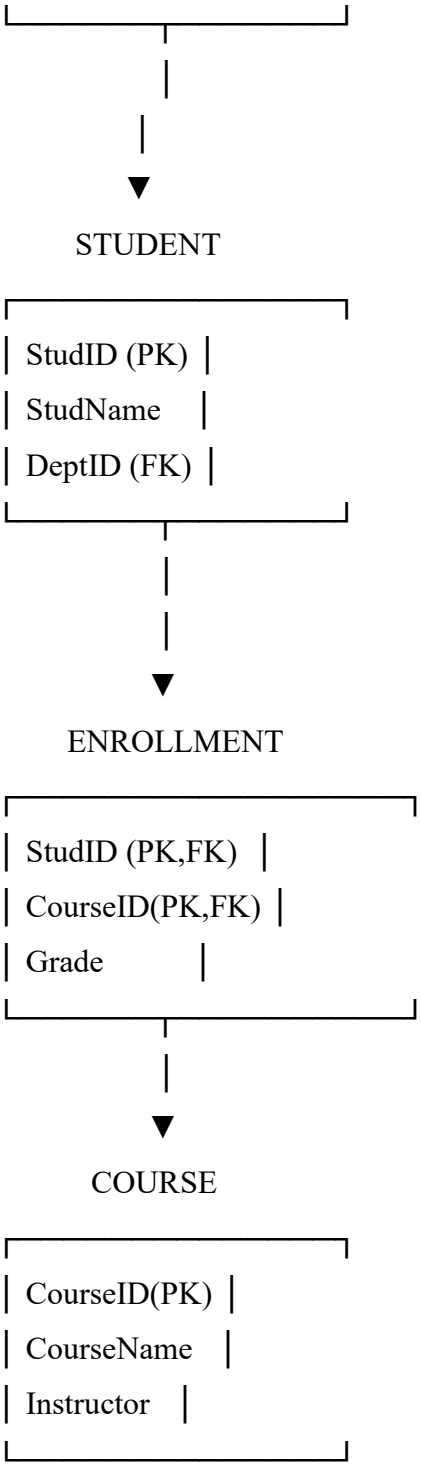
ENROLLMENT(StudID, CourseID, Grade)

- Primary Key: (StudID, CourseID)
- Foreign Keys: StudID, CourseID

7. Final Relationship Structure

DEPARTMENT

```
┌──────────────────┐
│ DeptID (PK)      │
│ DeptName         │
```



8. Summary

Normal Form	Problem Removed	Result
1NF	Non-atomic/repeating values	Atomic attributes

Normal Form	Problem Removed	Result
2NF	Partial dependencies	STUDENT, COURSE, ENROLLMENT
3NF	Transitive dependency	DEPARTMENT separated

Final 3NF Relations

STUDENT(StudID, StudName, DeptID)

DEPARTMENT(DeptID, DeptName)

COURSE(CourseID, CourseName, Instructor)

ENROLLMENT(StudID, CourseID, Grade)

Conclusion:

The original college schema suffers from insertion, deletion, and update anomalies because student, department, course, and enrollment information is stored together. Decomposing it into the four relations above removes partial and transitive dependencies, reduces redundancy, and produces a 3NF database design.

10.Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

Sure. Since no specific relation or FDs were included in the question, here is a complete example showing how to compute attribute closure and find all candidate keys.

Example

Consider:

R(A, B, C, D, E)

Functional Dependencies:

1. A → BC
2. BC → D
3. D → E

We need to compute attribute closures and determine all candidate keys.

Step 1: Compute A^+

Start with:

$A^+ = \{A\}$

Using A → BC:

$A^+ = \{A, B, C\}$

Using $BC \rightarrow D$:

$A^+ = \{A, B, C, D\}$

Using $D \rightarrow E$:

$A^+ = \{A, B, C, D, E\}$

Therefore:

$A^+ = \{A, B, C, D, E\}$

Since A determines all attributes, A is a candidate key.

Step 2: Compute B^+

Start:

$B^+ = \{B\}$

No FD has only B on the left side.

Therefore:

$B^+ = \{B\}$

B is not a candidate key.

Step 3: Compute C^+

Start:

$C^+ = \{C\}$

No FD can be applied.

Therefore:

$C^+ = \{C\}$

C is not a candidate key.

Step 4: Compute D^+

Start:

$D^+ = \{D\}$

Using $D \rightarrow E$:

$D^+ = \{D, E\}$

Therefore:

$D^+ = \{D, E\}$

D is not a candidate key.

Step 5: Compute E^+

Start:

$E^+ = \{E\}$

No FD can be applied.

Therefore:

$E^+ = \{E\}$

E is not a candidate key.

Closure Summary

Attribute Set	Closure	Candidate Key?
A	{A, B, C, D, E}	Yes
B	{B}	No
C	{C}	No
D	{D, E}	No
E	{E}	No

Therefore:

Candidate Key = {A}

General Method for Finding Candidate Keys

To find candidate keys:

1. Start with an attribute or combination of attributes.
2. Calculate its closure (X^+).
3. If X^+ contains all attributes of R, X is a superkey.
4. Check whether any attribute can be removed while still determining all attributes.
5. If no attribute can be removed, X is a candidate key.

Important Formula

For relation R:

If $X^+ = R$, then X is a superkey.

If X is a superkey and no proper subset of X is a superkey:

X is a candidate key.

If you provide the specific relation and FDs from your question, I can compute all closures and all candidate keys step-by-step for that exact problem.